



**UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO FACULTAD
DE CIENCIAS DE LA INGENIERÍA CARRERA DE SOFTWARE**

Tema:

PRÁCTICA EXPERIMENTAL 1 INGENIERÍA DE REQUISITOS

Asignatura:

Ingeniería De Requerimientos

Integrantes:

Mendoza Palma Allan

Nivela Saavedra Jose Ricardo

Velez Vines Jostin Gustavo

Carrera:

Software

Curso:

4to "B"

Docente:

Ing.Guerrero Ulloa Gleiston Ciceron

Fecha:

24 / Mayo / 2026

Índice

SECCIÓN 0 - INFORMACIÓN DEL PFC	4
0.1 Integrantes y Roles.....	4
0.2 Descripción el sistema del PFC	4
0.3 Tipo de Sistema y Justificación	5
SECCIÓN 2 - FUNDAMENTO TEÓRICO.....	5
2.1 Ingeniería de Requisitos.....	5
2.1.1 Definición de Ingeniería de Requisitos.....	5
2.1.2 Objetivos de la IR	5
2.1.3 Importancia de la IR en proyectos de software.....	6
2.2 Requisito de Software	6
2.2.1 Definición de requisito de software	6
2.2.2 Cadena necesidad → meta → requisito	7
2.2.3 Características de un buen requisito	7
2.2.4 Requisitos verificables y cuantificables.....	8
2.3 Tipos de Requisitos.....	8
2.3.1 Requisitos funcionales	8
2.3.2 Requisitos no funcionales	9
2.3.3 Requisitos de producto.....	9
2.3.4 Requisitos de proceso	10
2.3.5 Propiedades emergentes.....	10
2.3.6 Requisitos cuantificables	10
2.3.7 Restricciones	11
2.4 Proceso de Ingeniería de Requisitos	11
2.4.1 Definición del proceso de IR	11
2.4.2 Modelos de proceso	12
2.4.3 Actores del proceso.....	12
2.4.4 Actividades del proceso	12
2.5 Framework de Pohl y Contexto del Sistema.....	13
2.5.1 Actividades Core.....	13
2.5.2 Actividades transversales.....	14

2.5.3 Artefactos	14
2.5.4 Contexto del sistema	14
2.5.5 Ocho perspectivas de Pohl	15
2.6 Tipos de Sistemas y Visión del Proyecto.....	15
2.6.1 Sistemas de información	15
2.6.2 Sistemas embebidos	16
2.6.3 Sistemas software-intensivos	16
2.6.4 Sistemas autoadaptativos	17
2.6.5 Límite del sistema	18
2.6.6 Visión del proyecto	18
SECCIÓN 3 - ARTEFACTOS DEL PROCEDIMIENTO.....	19
3.1 Tabla de Tipos de Requisitos	19
3.2 Perspectivas de Contexto	20
3.3 Declaración de Visión.....	21
3.4 Límite del Sistema	22
3.5 Mapa Conceptual Integrador.....	23
4 CONCLUSIONES	24
Conclusión 1	24
Conclusión 2	24
Conclusión 3	24
Conclusión 4	24
Conclusión 5	25
5 REFERENCIAS.....	25
6ANEXOS	26
7 DECLARACIÓN DE USO DE LA IA.....	31

SECCIÓN 0 - INFORMACIÓN DEL PFC

0.1 Integrantes y Roles

Integrantes	Cédula	Correo Institucional	Rol
Mendoza Palma Allan Jeremy	0956082309	amendozap10@uteq.edu.ec	Líder, Documentador, Investigador C
Nivela Saavedra Jose Ricardo	1250450333	jniveles2@uteq.edu.ec	Investigador A
Velez Vines Jostin Gustavo	1316917838	jvelezv10@uteq.edu.ec	Investigador B

0.2 Descripción el sistema del PFC

La plataforma propuesta es un marketplace universitario orientado a la compra, venta e intercambio de productos y recursos relacionados con la vida académica. El sistema permitirá publicar materiales como libros, apuntes, uniformes, calculadoras, accesorios y otros artículos de uso estudiantil, además de facilitar la búsqueda por categorías, el contacto entre usuarios y la gestión básica de publicaciones. Su propósito es centralizar en un solo espacio digital las transacciones que hoy se realizan de forma dispersa en grupos de mensajería o redes sociales, mejorando la organización, la confianza y la trazabilidad de las ofertas.

A continuación, se identifican los stakeholders principales del proyecto y se clasifican según su nivel de poder e interés en la evolución del sistema.

Tabla 1. Matriz Poder/Interés de Stakeholders

Código	Stakeholder	Rol en el proyecto	Poder	Interés	Estrategia
STK-01	Estudiantes compradores y vendedores	Usuarios finales que publican, buscan y realizan transacciones	Bajo	Alto	Mantener informados
STK-02	Administrador del marketplace	Modera publicaciones, usuarios y reportes	Alto	Alto	Gestionar de cerca
STK-03	Autoridades universitarias	Definen normas institucionales y aprueban el uso del sistema	Alto	Medio	Mantener satisfechas
STK-04	Equipo de desarrollo y soporte TI	Diseña, implementa y mantiene la plataforma	Alto	Alto	Gestionar de cerca
STK-05	Biblioteca universitaria	Entidad de apoyo académico y difusión.	Medio	Alto	Puede apoyar la difusión del sistema y promover el intercambio de material académico entre estudiantes.

0.3 Tipo de Sistema y Justificación

El proyecto se clasifica principalmente como un sistema de información software-intensivo. Se considera un sistema de información porque administra datos estructurados de usuarios, productos, mensajes, publicaciones y transacciones, permitiendo operaciones de consulta, registro, actualización y seguimiento. Al mismo tiempo, es software-intensivo porque el valor principal del proyecto depende de las funcionalidades de la plataforma web: búsqueda, publicación, reputación, mensajería, control de publicaciones y apoyo a la interacción entre estudiantes. No se trata de un sistema embebido ni de control físico, sino de una solución digital centrada en el manejo de información y en la experiencia de uso.

SECCIÓN 2 - FUNDAMENTO TEÓRICO

2.1 Ingeniería de Requisitos

2.1.1 Definición de Ingeniería de Requisitos

“La ingeniería de requisitos es una función interdisciplinaria que media entre los dominios del adquirente y el proveedor para establecer y mantener los requisitos que debe cumplir el sistema, software o servicio de interés. La ingeniería de requisitos se ocupa de descubrir, educir, desarrollar, analizar, verificar, validar, comunicar, documentar y gestionar requisitos [1]”.

La ingeniería de requisitos es una disciplina que da la garantía de que todas las personas involucradas en la creación de un sistema tengan un entendimiento común de lo que se tiene planeado construir. Si esta disciplina no existiera los proyectos de software estarían repletos de errores que cuestan dinero y tiempo en corregirse debido a la mala comunicación entre las partes involucradas.

Ejemplo: En el desarrollo del marketplace universitario, la ingeniería de requisitos permite que el equipo entienda que los estudiantes no solo quieren publicar productos, sino que también necesitan confiar en los vendedores. A través de técnicas de educación, el equipo descubre que los estudiantes valoran ver la reputación del vendedor antes de comprar. Esta necesidad se documenta como un requisito funcional y se valida con los propios estudiantes antes de escribir una sola línea de código, evitando así construir un sistema sin mecanismos de confianza que nadie usaría.

2.1.2 Objetivos de la IR

"El objetivo de la IR es especificar y gestionar los requisitos de los sistemas de tal manera que los sistemas implementados y desplegados satisfagan los deseos y necesidades de las partes interesadas [2]"

El objetivo real de la ingeniería de requisitos es asegurar que el software final resuelva los problemas de los usuarios. Esto quiere decir que no se trata solo de escribir requisitos, sino que también gestionar los cambios que surgen durante el desarrollo. Un buen proceso de ingeniería de requisitos disminuye el riesgo de construir un sistema equivocado que no solucione el problema principal.

Ejemplo: El objetivo de la IR en el marketplace es garantizar que los estudiantes puedan vender, comprar e intercambiar productos de manera fácil y segura. Para lograrlo, el equipo especifica requisitos como "el sistema permitirá buscar productos por categoría" y "el sistema mostrará la reputación del vendedor". Cuando los estudiantes solicitan agregar un chat en tiempo real, la gestión de requisitos evalúa el impacto en el costo y el cronograma, y decide priorizarlo frente a otras funcionalidades menos valoradas, como los filtros avanzados de búsqueda. Importancia de la IR en proyectos software.

2.1.3 Importancia de la IR en proyectos de software

"Una IR adecuada añade valor en el proceso de desarrollo y evolución de un sistema: • Reduciendo el riesgo de desarrollar un sistema incorrecto • Mejorando la comprensión del problema • Siendo la base para estimar el esfuerzo y el coste del desarrollo • Siendo un prerrequisito para probar el sistema [2]"

La importancia de la ingeniería de requerimientos reside en que con una buena aplicación de esta la comprensión que tendrán los desarrolladores sobre los deseos del cliente será mejor lo cual conlleva a la creación de un software que cumple con las expectativas, además un buen proceso de ingeniería de requerimiento ayuda a la estimación del tiempo y esfuerzo que se pondrá en un proyecto.

Ejemplo: En el Marketplace, si el equipo no hace un buen proceso de recolección de requisitos podrían crear un proyecto que no satisfaga las necesidades de los estudiantes que buscan vender, comprar o intercambiar objetos de manera segura dentro de la universidad.

2.2 Requisito de Software

2.2.1 Definición de requisito de software

"Formalmente, un requisito de software ha sido definido como: una condición o capacidad necesaria para que un usuario resuelva un problema o alcance un objetivo; una condición o capacidad que debe ser cumplida o poseída por un sistema o componente del sistema para satisfacer un contrato, estándar, especificación u otro documento formalmente impuesto [3]"

Un requisito de software es una necesidad del usuario dicha de manera clara, sin ambigüedades y de forma que pueda ser verificada. No es una idea vaga como "El programa sea rápido" es una declaración concreta "El sistema debe cargar los resultados

de la búsqueda en menos de 2 segundos” este ejemplo muestra un requisito claro y cuantificable.

Ejemplo: Los estudiantes probablemente necesiten comprar o vender productos de semestres anteriores como uniformes que ya no son de tallas usables o libros comprados que no se utilizaron, entonces el programa deberá permitirles publicar estos productos con información necesaria para vender como lo sería título, descripción, precio y al menos una fotografía.

2.2.2 Cadena necesidad → meta → requisito

"Ocurre un problema cuando las partes implicadas no están satisfechas con la situación tal como es. Los requisitos capturan lo que las partes implicadas necesitan para eliminar el problema o para hacerlo más fácil. Un sistema socio - técnico que satisface estos requisitos constituye una solución. Los problemas, requisitos y soluciones no ocurren necesariamente en este orden. Las ideas de solución pueden crear necesidades de usuario que deben ser elaboradas como requisitos e implementadas en una solución real [2]"

Los analistas probablemente necesiten rastrear el camino desde el problema original del usuario hasta la solución técnica para no terminar construyendo funcionalidades inútiles que no resuelven nada, entonces esta cadena de necesidad–meta–requisito deberá permitirles ver cómo se comporta el sistema con información clave para la trazabilidad cuando cambie el negocio como lo sería identificar qué requisitos específicos y qué partes del código se van a ver afectados.

Ejemplo: Problema (insatisfacción): Los estudiantes gastan un dineral comprando libros nuevos cada semestre, a pesar de que los chicos de semestres superiores ya no usan los suyos y los tienen acumulados. Meta: Facilitar la reutilización de estos materiales de estudio dentro de la misma comunidad universitaria. Requisito derivado: El sistema debe permitir a los alumnos publicar sus productos usados detallando el precio y el estado en el que están.

2.2.3 Características de un buen requisito

“Un requisito debe cumplir criterios de calidad específicos para ser considerado un buen requisito. En la IR moderna, con enfoques orientados al valor (Principio 1 en 2), el grado de cumplimiento de un criterio de calidad se corresponderá con valor creado por este requisito. Esto significa que los requisitos no tienen que satisfacer completamente todos los criterios de calidad — pero, cuanto más alto sea el valor de un requisito, más relevantes son sus criterios de calidad asociados, con el fin de reducir el riesgo de fracaso. La adecuación y la comprensibilidad son los criterios de calidad más importantes para los requisitos individuales. Sin ellos, un requisito es inútil o incluso perjudicial, independientemente del cumplimiento de todos los demás criterios. Criterios de calidad para requisitos individuales: ▪ Adecuado (describe necesidades reales y acordadas con las

partes interesadas) ▪ Necesario ▪ No ambiguo ▪ Completo (auto-contenido) ▪ Comprensible ▪ Verificable [2]”

Un buen requisito no es solo una frase que suena bonita o formal para cumplir. Al final del día, lo que importa es que le sirva al equipo y que no deje espacio a dudas, porque si no, el cliente entiende una cosa y el programador termina haciendo algo totalmente diferente. Además, tiene que ser verificable, o sea, que de alguna forma puedas hacer una prueba y decir "sí, el sistema cumple" o "no, le falta". Por último, tiene que nacer de una necesidad real de los estudiantes y no de un simple invento que a nadie le importa.

Ejemplo: Poner en el documento que "el sistema debe ser rápido" es un error clásico. Eso no le dice nada a nadie, porque rápido para mí puede ser un segundo y para otro usuario son cinco. Sería mejor poner algo como: "El sistema cargará la lista de productos en menos de 2 segundos desde el internet de la U". Ahí ya es medible con un cronómetro, está claro y soluciona la molestia real del alumno de quedarse esperando a que cargue la página.

2.2.4 Requisitos verificables y cuantificables

"Un requisito debe ser comprobable (cuantificado), lo que significa que el cumplimiento o incumplimiento puede demostrarse claramente [3]"

Si a la gente de control de calidad le das requisitos que no se pueden probar, les complicas la vida. Ellos necesitan datos objetivos para poder armar sus casos de prueba y testear el software antes de lanzarlo. Por eso es clave meter números. La cuantificación ayuda a bajar a la tierra palabras abstractas como "seguro", "fácil" o "rápido" y convertirlas en métricas reales, como un 99.9% de disponibilidad. Si quieres guiarte, la norma ISO 25010 da buenas ideas de qué cosas medir.

Ejemplo: Si al equipo le pides "haz el marketplace seguro", es muy ambiguo. Es mil veces mejor desglosarlo en cosas que sí se puedan probar de una. Por ejemplo, exigir el correo y la clave para transaccionar, mandar los datos de tarjetas a un proveedor certificado como PCI-DSS para no guardar nada en nuestra base de datos, y que el sistema guarde en un log la IP de cada intento fallido de login.

2.3 Tipos de Requisitos

2.3.1 Requisitos funcionales

"Los requisitos funcionales se refieren a un resultado o comportamiento que será proporcionado por una función de un sistema. Esto incluye los requisitos de datos o la interacción de un sistema con su entorno [2]"

Básicamente, son las acciones que hace el sistema cuando el usuario interactúa con él. Es la parte más visible del software, las pantallas y botones que los estudiantes van a usar en el día a día para resolver sus tareas. Al programador le sirven un montón. Con esta lista de

funciones es que se calcula el esfuerzo de desarrollo y cuánto va a tomar el diseño, el código y las pruebas.

Ejemplo: Por ejemplo, si el alumno elige la categoría "libros" o "uniformes", la pantalla debe filtrar y mostrar solo eso. También, al darle clic en "Publicar", el sistema tiene que guardar el anuncio con su título, precio, fotos y si es nuevo o usado. Y si alguien compra, el programa debe generar la transacción, bajar el stock y avisarle por correo al vendedor.

2.3.2 Requisitos no funcionales

“Los requisitos no funcionales limitan de alguna manera las tecnologías que se utilizarán en la implementación: ¿Qué plataforma(s) informática(s)? ¿Qué motor(es) de base de datos? ¿Qué grado de precisión deben tener los resultados? ¿Con qué rapidez deben presentarse los resultados? ¿Cuántos registros de un determinado tipo deben almacenarse? Algunos requisitos no funcionales pueden estar relacionados con el funcionamiento del software [3]”.

Aquí hablamos más del "cómo" funciona el sistema y no tanto del "qué" hace. Son esas cualidades que el software debe tener y que el usuario no lo ve directamente, como la velocidad, la seguridad o que aguante cuando se conecten muchos alumnos a la vez. Aunque el usuario no los ve como un botón físico, si fallan, la experiencia es pésima y la página se vuelve inusable. Además, suelen ser transversales, lo que significa que afectan a toda la plataforma por igual y no a una sola pantalla.

Ejemplo:

Rendimiento: El catálogo no puede tardar más de 3 segundos en cargar si hay menos de 100 productos en la categoría.

Disponibilidad: En semanas de exámenes finales (junio y diciembre), la web debe estar activa el 99.5% del tiempo.

Los datos personales como nombres o teléfonos tienen que viajar y guardarse cifrados con TLS 1.3 y AES-256.

2.3.3 Requisitos de producto

“Los requisitos de un producto de software especifican la forma, el ajuste o la función que se espera del software [3]”.

Estos se enfocan netamente en cómo debe ser el software ya terminado y entregado. Es lo que define las características del producto final, es decir, lo que vas a ver operando en producción. El equipo de testing revisa esto al final con pruebas directas y mirando los manuales de usuario para darle el visto bueno.

Ejemplo: Para el marketplace, el producto final tiene que guardar el historial de compras por un año. También debe dejar que te descargues los comprobantes de compra en PDF desde el perfil. Otra cosa es el panel de administración, que debe mostrar gráficos con la gente que se registra al día y los productos denunciados.

2.3.4 Requisitos de proceso

"Los requisitos del proyecto —también llamados requisitos de proceso o, a veces, requisitos de negocio— restringen el proyecto que construye el software [3]".

A diferencia de los anteriores, estos no definen qué hace el software, sino cómo debe trabajar el equipo para construirlo. Son límites impuestos para la gestión del proyecto, ya sea por metodologías, contratos o estándares de la organización. Sirven para que el desarrollo no sea un caos y se cumpla con la calidad que pide el cliente.

Ejemplo: Nadie sube código a la rama principal (main) sin que otro programador revise y apruebe su Pull Request. Además, se trabajará con sprints de dos semanas, mostrando avances reales a los profesores o clientes al final de cada ciclo. Y antes de cada despliegue, el login y las transacciones deben pasar pruebas de seguridad automatizadas

2.3.5 Propiedades emergentes

"Una propiedad emergente es una propiedad de un sistema de sistemas que no es deducible de las propiedades de los sistemas constituyentes individuales [4]"

Son características que no se notan al revisar una sola función o un botón por separado. Aparecen recién cuando todo el sistema trabaja en conjunto y se ve el comportamiento global de la plataforma en producción. Hay que vigilarlas de cerca porque pueden jugar a favor o en contra, y muchas veces de esto depende que el proyecto triunfe o fracase.

Ejemplo: La "confianza" en el sitio es una propiedad emergente. No la logras solo con poner un botón; nace de que funcione bien el sistema de calificaciones, de que el historial de ventas sea transparente, de que las fotos sean reales y de que el soporte responda rápido. Si la búsqueda es lenta o hay estafas, esa confianza se rompe por completo, aunque el botón de publicar funcione perfecto.

2.3.6 Requisitos cuantificables

"Los requisitos de calidad pueden cuantificarse utilizando un modelo de calidad como el estándar ISO 25010, que proporciona características y subcaracterísticas que se relacionan con las propiedades de calidad de los productos [5]"

Es ponerles números claros a los requisitos de calidad para no andar con rodeos ni malentendidos. Se trata de quitar los adjetivos subjetivos como "rápido" o "eficiente" y poner datos que se puedan medir. Así el equipo sabe qué tiene que mejorar realmente.

También ayuda a tomar decisiones técnicas complicadas, como, por ejemplo, sacrificar un poco de velocidad a cambio de meter más cifrado y seguridad.

Ejemplo: Usando métricas de la ISO 25010, definimos que el backend procese las búsquedas en menos de 500 milisegundos, aunque haya 10,000 productos guardados. También que aguante 1,000 usuarios concurrentes en hora pico sin pasar de los 2 segundos de respuesta, y asegurar que el 99.5% de las compras terminen sin errores en el servidor.

2.3.7 Restricciones

"Una restricción es una limitación impuesta externamente sobre el sistema, su diseño, implementación o sobre el proceso utilizado para desarrollar o modificar un sistema. Una restricción es un factor que se impone a la solución por fuerza o compulsión y puede limitar o modificar el diseño [1]"

Son los límites externos con los que el equipo tiene que lidiar obligatoriamente por fuerza mayor. No añaden valor al negocio ni nuevas funciones a la aplicación, pero reducen tus opciones al momento de diseñar la solución técnica. Vienen de la tecnología que ya tiene la organización, de temas legales o de políticas internas que casi nunca se pueden negociar.

Ejemplo: La autenticación se tiene que amarrar al directorio activo de la institución; no se puede inventar un login aparte.

2.4 Proceso de Ingeniería de Requisitos

2.4.1 Definición del proceso de IR

“La ingeniería de requisitos es un enfoque sistemático y disciplinado para la especificación y gestión de requisitos con los siguientes objetivos: conocer los requisitos relevantes, lograr un consenso entre las partes interesadas sobre estos requisitos, documentarlos de acuerdo con estándares determinados y gestionarlos sistemáticamente [6].”

La ingeniería de requisitos no es una actividad improvisada, sino que sigue un método ordenado y disciplinado para evitar errores costosos. Su objetivo principal es garantizar que todas las personas involucradas en el proyecto entiendan y estén de acuerdo con lo que se va a construir. Además, se encarga de que toda esta información quede registrada de manera ordenada y se pueda gestionar a lo largo del tiempo, incluso cuando surjan cambios.

Ejemplo: En el proyecto del marketplace universitario, el proceso comienza entrevistando a estudiantes para saber qué necesitan (conocimiento de requisitos). Luego, se escribe una lista clara como “el sistema debe permitir subir una foto del libro” (documentación). Finalmente, se reúne a vendedores y compradores para que acuerden si la publicación debe ser gratuita o con comisión (consenso), y todo esto se registra formalmente.

2.4.2 Modelos de proceso

“En un proceso lineal, los requisitos se especifican por adelantado en una sola fase del proceso. En un proceso iterativo, los requisitos se especifican de manera incremental, comenzando con los objetivos generales y algunos requisitos iniciales, y luego agregando o modificando requisitos en cada iteración [6].”

El modelo en cascada funciona de forma lineal: primero se escriben todos los requisitos, luego se diseña, luego se programa, y así sucesivamente, sin volver atrás. El modelo iterativo reconoce que los requisitos no se conocen todos al principio, por lo que se construye el sistema en ciclos repetitivos, mejorándolo cada vez. El modelo ágil entrega pequeñas partes funcionales del sistema en ciclos muy cortos (semanas), permitiendo ajustes rápidos basados en la retroalimentación de los usuarios.

Ejemplo: En cascada se definirían todos los requisitos de pago antes de escribir una sola línea de código. En iterativo se construiría primero una versión simple con búsqueda de libros, luego en otro ciclo se añadiría el carrito de compras, y después el sistema de pagos. En ágil se lanza primero la función “ver catálogo” en 2 semanas, luego “añadir al carrito” en la siguiente iteración, y los estudiantes pueden probar y sugerir cambios inmediatamente.

2.4.3 Actores del proceso

“Una parte interesada de un sistema es una persona o una organización que tiene una influencia (directa o indirecta) sobre los requisitos del sistema [6].”

Los actores son todas las personas, grupos u organizaciones que pueden afectar o ser afectados por el sistema que se va a construir. Algunos actores tienen una influencia directa, como los usuarios finales que operarán el sistema día a día. Otros tienen una influencia indirecta, como los reguladores o la dirección de la universidad, que imponen restricciones sin usar el sistema directamente.

Ejemplo: Los estudiantes son los actores principales, tanto como vendedores (publican libros) como compradores (buscan y adquieren materiales). El administrador del marketplace modera las publicaciones y resuelve disputas. La universidad es un actor indirecto que impone reglas (por ejemplo, prohibir la venta de apuntes falsos). El equipo de desarrollo (programadores y analistas) también es un actor porque sus capacidades técnicas limitan o habilitan ciertas funciones.

2.4.4 Actividades del proceso

“Las cuatro actividades centrales para alcanzar estos fines son las siguientes: Obtención, Documentación, Validación, negociación y Gestión [6].”

La Elicitación consiste en obtener la información desde las fuentes originales (usuarios, documentos, sistemas existentes) mediante entrevistas, observación o análisis de

documentos. La Documentación es el registro formal de esa información en un formato que pueda ser entendido y consultado por todos los involucrados. La Validación y negociación implica revisar que los requisitos escritos sean correctos y resolver los conflictos cuando diferentes usuarios piden cosas opuestas. La Gestión se encarga de controlar los cambios que inevitablemente surgen durante el proyecto, manteniendo la trazabilidad y el control de versiones.

Ejemplo: Elicitación: Entrevistar a un alumno de ingeniería sobre cómo busca calculadoras usadas. Documentación: Escribir la regla “el buscador debe filtrar por materia y precio”. Validación: Mostrar esa regla al alumno y que él confirme “sí, eso es lo que necesito”. Negociación: Cuando un vendedor quiere eliminar valoraciones negativas pero un comprador quiere ver el historial real, se negocia mostrando la valoración, pero permitiendo respuesta del vendedor. Gestión: Cuando la universidad cambia su política de privacidad, se actualizan los requisitos de protección de datos.

2.5 Framework de Pohl y Contexto del Sistema

2.5.1 Actividades Core

“Elicitación: Durante la obtención de requisitos, se utilizan diferentes técnicas para obtener requisitos de las partes interesadas y otras fuentes... Documentación: Durante la documentación, los requisitos obtenidos se describen adecuadamente... Validación y negociación: Para garantizar que se cumplan los criterios de calidad predefinidos, los requisitos documentados deben validarse y negociarse desde el principio... Gestión: La gestión de requisitos es ortogonal a todas las demás actividades [6].”

Las actividades Core son el corazón de la ingeniería de requisitos y se aplican en todo proyecto, sin importar su tamaño o dominio. La elicitación busca sacar a la luz lo que los usuarios realmente necesitan, a menudo cuando ellos mismos no saben expresarlo claramente. La documentación transforma ese conocimiento tácito en algo tangible y verificable. La validación y negociación aseguran que lo escrito sea correcto y que todos estén de acuerdo antes de empezar a construir. La gestión mantiene todo organizado cuando el proyecto evoluciona y cambian las prioridades.

Ejemplo: Elicitación: Un estudiante sugiere “sería bueno tener un chat interno para negociar el precio”. Documentación: Se especifica “el sistema almacenará mensajes cifrados y notificará al comprador en menos de 5 segundos”. Validación: El estudiante revisa y dice “sí, eso es exactamente lo que quiero”. Negociación: Los vendedores quieren eliminar valoraciones negativas, pero los compradores quieren ver la historia real; se negocia mostrando la valoración, pero permitiendo respuesta pública del vendedor. Gestión: Cuando luego se pide agregar notificaciones push, se registra el cambio y se actualiza la documentación.

2.5.2 Actividades transversales

“Validación y negociación... Gestión” (actividades que aplican a todo el proceso) [6].”

Las actividades transversales son aquellas que no se realizan una sola vez al inicio, sino que acompañan a todo el proyecto de principio a fin. La validación no es un evento único, sino que debe repetirse cada vez que se añaden o modifican requisitos para detectar errores temprano. La gestión tampoco es un paso aislado, sino un proceso continuo que registra cada cambio, cada decisión y cada versión de los requisitos a lo largo del tiempo. Su objetivo es mantener la calidad y el control durante toda la vida del sistema.

Ejemplo: Revisar la especificación “el sistema debe enviar email de confirmación” y detectar que falta qué pasa si el servidor de email falla (debe reintentar o mostrar error). Gestión: Cuando la universidad cambia su política de privacidad (contexto), el gestor de requisitos actualiza los documentos de seguridad del sistema. Además, cada vez que se aprueba un cambio, se registra la nueva versión del requisito afectado.

2.5.3 Artefactos

“Un producto de trabajo es un resultado intermedio o final, que se genera en un proceso de trabajo y que se registra. Hay una variedad de productos de trabajo en RE, que van, por ejemplo, desde bocetos gráficos de corta duración, pasando por colecciones en evolución de historias de usuarios, hasta documentos de especificación de requisitos contractualmente vinculantes con cientos de páginas que se entregan oficialmente [7].”

Los artefactos son los productos tangibles que se generan al hacer ingeniería de requisitos, y pueden tener diferente nivel de formalidad y duración. Algunos artefactos son temporales, como los bocetos dibujados en una pizarra durante una reunión, que se desechan después de usarlos. Otros son evolutivos, como una lista de historias de usuario que se va actualizando durante todo el proyecto. Los más formales son los documentos de especificación que pueden tener valor contractual y deben conservarse durante años.

Ejemplo: Artefacto temporal: Un boceto en una pizarra de cómo debería verse la pantalla de búsqueda de libros. Artefacto en evolución: El “product backlog” con historias de usuario como “como estudiante quiero filtrar por precio”. Artefacto duradero: La especificación de requisitos del sistema que se entrega formalmente al equipo de desarrollo y se usa como base contractual. Otro artefacto: El glosario que define qué significa “libro en buen estado” para todos los involucrados.

2.5.4 Contexto del sistema

“El contexto del sistema es la parte del entorno del sistema que es relevante para la definición y la comprensión de los requisitos de un sistema a desarrollar [6].”

El contexto del sistema incluye todo aquello que rodea al sistema, pero no es parte de él, y que sin embargo influye en sus requisitos. Esto puede ser personas (usuarios), otros

sistemas (bases de datos externas, APIs), procesos (cómo se hace el trabajo actualmente) o documentos (leyes, normas, políticas universitarias). Si se ignora alguno de estos elementos, es muy probable que los requisitos estén incompletos y el sistema falle en la práctica. Por eso, una de las primeras tareas del ingeniero de requisitos es entender y documentar este contexto.

Ejemplo: No solo importa el código del marketplace. Influye el proceso de negocio actual (los estudiantes se organizan por grupos de WhatsApp para vender libros). Influye el sistema externo de pasarela de pagos (Mercado Pago o Stripe). Influye el documento de normas de la universidad (prohíbe la venta de apuntes con derechos de autor). Influye el estudiante (usuario) y su comportamiento de búsqueda. Todos estos elementos están en el contexto y afectan los requisitos.

2.5.5 Ocho perspectivas de Pohl

“Al documentar requisitos sobre la base de modelos, normalmente se distinguen tres tipos de perspectivas: datos, función y comportamiento [6].”

Las perspectivas son diferentes formas de mirar un mismo sistema para asegurarse de que no se olvida ningún detalle importante. La perspectiva de datos se enfoca en qué información maneja el sistema, cómo se estructura y cómo se relaciona. La perspectiva de función describe qué hace el sistema, paso a paso, transformando entradas en salidas. La perspectiva de comportamiento explica cómo reacciona el sistema ante eventos externos, cambiando de estado según las condiciones. Al combinar estas tres visiones, se obtiene una comprensión completa y sin omisiones del sistema.

Ejemplo: Perspectiva de datos: Los atributos del libro (ISBN, título, autor, precio, estado, fotos). Perspectiva de función: El proceso de “publicar un libro” incluye: subir foto, ingresar precio, seleccionar categoría, confirmar publicación. Perspectiva de comportamiento: El sistema tiene estados: “libro disponible”, “libro reservado” (cuando alguien inicia compra), “libro vendido” (cuando se completa el pago). Si un libro está vendido, el sistema no permite otra compra.

2.6 Tipos de Sistemas y Visión del Proyecto

2.6.1 Sistemas de información

“Una base de datos es una colección de elementos de datos relacionados, recopilados específicamente para su uso por una o más aplicaciones y almacenados en un formato organizado para un acceso fácil y rápido [8].”

Los sistemas de información están diseñados principalmente para gestionar grandes volúmenes de datos estructurados de manera eficiente. Su función central es permitir a los usuarios crear, leer, actualizar y eliminar información (operaciones CRUD) de forma rápida y segura. El valor de estos sistemas no está en los algoritmos complejos, sino en la

calidad y disponibilidad de los datos que almacenan y recuperan. Por eso, aspectos como la integridad referencial, la normalización y el rendimiento de las consultas son críticos en este tipo de sistemas.

Ejemplo: El marketplace universitario es un sistema de información porque gestiona datos de estudiantes (nombre, correo institucional, carrera), productos (título del libro, ISBN, precio, estado de conservación, fotos) y transacciones (fecha de compra, método de pago, dirección de entrega). La base de datos relacional almacena estos registros y permite consultas como “mostrar todos los libros de cálculo disponibles que cuesten menos de 20 euros”. También permite generar reportes como “los 10 libros más vendidos del mes” o “los vendedores con mejores valoraciones”.

2.6.2 Sistemas embebidos

“Los sistemas integrados suelen ser sistemas heterogéneos. Los sistemas heterogéneos constan de varias unidades computacionales especializadas de diferentes tipos, como unidades de procesamiento gráfico (GPU) y procesadores de señales digitales (DSP), microcontroladores y procesadores periféricos [8].”

Los sistemas embebidos son dispositivos que contienen software integrado dentro de un hardware específico, diseñado para realizar una o pocas funciones concretas. A diferencia de una computadora personal, que puede ejecutar cualquier programa, un sistema embebido está dedicado a tareas muy particulares, como controlar un motor, leer sensores o procesar señales. El usuario final normalmente no interactúa con ellos como si fueran “computadoras”, sino a través del dispositivo físico que los contiene (un electrodoméstico, un automóvil, una calculadora). Su software suele ser crítico en tiempo real y consume muy poca energía.

Ejemplo: Aunque el marketplace principal es una aplicación web, podría interactuar con sistemas embebidos en el entorno universitario. Por ejemplo, un lector de códigos de barras conectado a una computadora en la cafetería de la facultad permite escanear el código de barras de un libro usado y buscar automáticamente su ficha en el marketplace. Otro ejemplo es una calculadora gráfica programable que, al conectarse al Wi-Fi de la universidad, sincroniza automáticamente los apuntes y ejercicios comprados en la plataforma. También un dispositivo IoT colocado en la biblioteca que avisa mediante un LED cuando un libro reservado ya está disponible para retirar.

2.6.3 Sistemas software-intensivos

“Existe una gama de sistemas creados por humanos, desde aquellos que utilizan poco o ningún software hasta aquellos en los que el software es el interés principal. Cuando el software es el sistema predominante o elemento de interés, se debe utilizar ISO/IEC/IEEE 12207 [9].”

Los sistemas software-intensivos son aquellos en los que el software no es un componente secundario, sino la parte principal que aporta el valor diferenciador del sistema. Sin el software, el sistema perdería casi toda su utilidad o simplemente no tendría razón de existir. Por esta razón, en estos sistemas la ingeniería de software adquiere una importancia crítica: los procesos de desarrollo, la arquitectura, las pruebas y el mantenimiento del código son el foco central. Ejemplos típicos son las aplicaciones móviles, las plataformas web, los sistemas de comercio electrónico y las redes sociales.

Ejemplo: El marketplace universitario es un sistema software-intensivo porque toda su funcionalidad principal reside en el software. El registro de usuarios con correo institucional, la publicación de libros con fotos y descripciones, el sistema de búsqueda con filtros, el carrito de compras, la pasarela de pagos, el sistema de calificaciones entre estudiantes... todo eso es software. El hardware (servidores en la nube, computadoras y teléfonos de los estudiantes) es solo el soporte físico para ejecutar ese software. Si se elimina el software, el marketplace simplemente deja de existir.

2.6.4 Sistemas autoadaptativos

“Un modelo es una representación de una parte de una realidad existente o de una parte de una realidad por crear. Los modelos ayudan a comprender las relaciones e interconexiones entre los requisitos y proporcionan una visión general de un conjunto de requisitos (Los sistemas autoadaptativos utilizan modelos dinámicos y machine learning para cambiar su comportamiento según el contexto) [6, 8].”

Los sistemas autoadaptativos son capaces de modificar su propio comportamiento en respuesta a cambios en el entorno o en los patrones de uso, sin necesidad de intervención humana directa. Para lograr esto, utilizan técnicas de aprendizaje automático (machine learning) que analizan grandes volúmenes de datos históricos y detectan patrones, tendencias o anomalías. Con esa información, el sistema reconfigura sus reglas internas, ajusta parámetros o sugiere acciones para optimizar su rendimiento o la satisfacción del usuario. De esta forma, el sistema “aprende” y mejora con el tiempo, adaptándose a necesidades que no se habían previsto inicialmente.

Ejemplo: El marketplace podría ser autoadaptativo de varias formas. Si detecta que muchos estudiantes buscan “Calculadora Texas TI-84” y no hay unidades disponibles, automáticamente envía una notificación a vendedores que hayan vendido ese modelo antes, sugiriéndoles que publiquen si tienen una para vender. También podría ajustar dinámicamente el precio sugerido de un libro basándose en la demanda histórica y la oferta actual. Además, podría recomendar libros personalizados a cada estudiante según su historial de compras, su carrera y los libros que otros estudiantes de la misma materia han adquirido.

2.6.5 Límite del sistema

“El límite del sistema separa el sistema a desarrollar de su entorno; es decir, separa la parte de la realidad que puede ser modificada o alterada por el proceso de desarrollo de aspectos del entorno que no pueden ser cambiados o alterados por el proceso de desarrollo [6].”

El límite del sistema es una línea conceptual que separa “lo que hace mi sistema” (lo que podemos diseñar, construir y modificar) de “lo que hacen los usuarios, otros sistemas o el entorno” (lo que no podemos cambiar). Todo lo que está dentro del límite es responsabilidad del equipo de desarrollo y puede ser modificado durante el proyecto. Todo lo que está fuera del límite debe ser aceptado como una condición dada, una restricción o un servicio provisto por terceros.

Ejemplo: El límite del sistema incluye la gestión de publicaciones, la búsqueda, el carrito de compras y la comunicación entre comprador y vendedor mediante mensajería interna. Si un vendedor publica un libro que no existe o un comprador reporta un fraude, el sistema puede detectar la denuncia, mostrar una advertencia y suspender temporalmente al usuario infractor (todo dentro del límite). Sin embargo, expulsar al alumno de la universidad o aplicar una sanción académica está fuera del límite del sistema: eso lo debe hacer la autoridad universitaria. El sistema solo puede notificar a la universidad, pero no ejecutar la sanción.

2.6.6 Visión del proyecto

“La visión del proyecto resume el propósito del proyecto de manera clara y concisa. Describe una perspectiva realista y atractiva de los resultados futuros del proyecto [8].”

La visión del proyecto es una declaración breve y poderosa que responde a la pregunta “¿por qué estamos construyendo este sistema?”. Una buena visión debe ser clara (que todos la entiendan), concisa (que se pueda recordar fácilmente), realista (alcanzable con los recursos disponibles) y motivadora (que inspire al equipo a dar lo mejor de sí). La visión actúa como la “estrella polar” que guía todas las decisiones técnicas y de negocio: si una funcionalidad no contribuye directamente a la visión, probablemente debe descartarse o posponerse. Además, la visión ayuda a priorizar esfuerzos y a mantener la coherencia del proyecto a lo largo del tiempo, incluso cuando cambian los miembros del equipo.

Ejemplo: “Crear el mercado de intercambio universitario más confiable de la universidad, reduciendo el costo de materiales para los estudiantes en un 40% para el año 2026”. Esta visión guía todas las decisiones: si alguien propone añadir funcionalidades sociales no relacionadas directamente con los objetivos académicos y comerciales del sistema, el equipo lo rechaza porque no reduce costos. Si se propone invertir en un sistema de verificación de identidad con carné universitario, se aprueba porque aumenta la confiabilidad. Si se plantea una gamificación con puntos y premios, se analiza si realmente

ayuda a reducir costos o es una distracción. La visión mantiene al equipo enfocado en lo que realmente importa.

SECCIÓN 3 - ARTEFACTOS DEL PROCEDIMIENTO

3.1 Tabla de Tipos de Requisitos

La siguiente tabla resume requisitos representativos del marketplace universitario, redactados con enfoque práctico y alineados con las categorías solicitadas en la guía.

Tabla 2. Tipos de requisitos aplicado al PFC

ID	Tipo	Descripción	MoSCoW	Fuente
BR-01	Negocio	El sistema deberá centralizar la compra, venta e intercambio de materiales académicos dentro de la comunidad universitaria.	Must	[1,6]
BR-02	Negocio	El sistema deberá reducir la dependencia de grupos informales dispersos para publicar y negociar productos.	Should	[6]
SH-01	Stakeholder	Permitir que los estudiantes verifiquen la reputación del vendedor antes de concretar una compra.	Must	[1,6]
SH-02	Stakeholder	El sistema deberá permitir que los estudiantes verifiquen la reputación del vendedor antes de concretar una compra.	Must	[1,6]
US-01	Usuario	El sistema deberá permitir el registro de usuarios mediante correo institucional.	Must	[1,9]
US-02	Usuario	El sistema deberá permitir buscar y filtrar productos por categoría, precio, estado y relevancia.	Must	[1,6]
FR-01	Funcional	El sistema deberá permitir a los usuarios crear publicaciones de productos con título, descripción, precio, estado, categoría y al menos una imagen.	Must	[1]
FR-02	Funcional	El sistema deberá permitir a los compradores contactar a los vendedores mediante mensajería interna.	Should	[1,6]
FR-03	Funcional	El sistema deberá permitir a los usuarios marcar un producto	Should	[1,6]

		como reservado o vendido después de llegar a un acuerdo.		
NFR-01	No funcional	El sistema deberá mostrar resultados de búsqueda en menos de 2 segundos para hasta 1.000 publicaciones activas.	Must	[5]
NFR-02	No funcional	El sistema deberá transmitir los datos personales utilizando TLS 1.3 y almacenar la información sensible cifrada en reposo.	Must	[5]
PERF-01	Rendimiento	El sistema deberá soportar al menos 500 usuarios concurrentes sin que el tiempo de respuesta supere los 3 segundos.	Should	[5]
IF-01	Interfaz	La interfaz deberá ser adaptable para navegadores móviles y de escritorio.	Must	[5]
CON-01	Restricción / Conformidad	El sistema deberá cumplir con las políticas universitarias y las regulaciones aplicables de protección de datos personales.	Must	[1,5]
RES-01	Restricción	El sistema deberá permitir el acceso únicamente mediante cuentas de correo institucional validadas por el dominio de la universidad.	Must	[1,5]

3.2 Perspectivas de Contexto

Para considerar el contexto del marketplace universitario de forma completa, se aplican las ocho perspectivas de Pohl. Esta lectura permite identificar no solo lo que el sistema hará, sino también cómo lo usa la comunidad, qué elementos del dominio lo rodean, qué aspectos técnicos lo limitan y qué oportunidades o riesgos emergen.

Tabla 3. Perspectivas de contexto

Perspectiva	3 preguntas clave	2 requisitos / restricciones emergentes
Uso	¿Quién publica y quién compra? ¿Qué tareas realiza cada usuario? ¿En qué momentos se usa más la plataforma?	Interfaz móvil sencilla; flujo rápido para publicar y reservar productos.
Sujeto	¿Qué objetos del dominio existen? ¿Cómo se clasifican los productos? ¿Qué atributos necesita cada publicación?	Catálogo con vocabulario controlado; campos obligatorios de estado, categoría y precio.

Sistema de TI	¿Qué módulos internos participan? ¿Qué integraciones necesita? ¿Cómo se intercambia la información?	APIs autenticadas; trazabilidad y registro de eventos críticos.
Desarrollo	¿Cómo se trabajará por iteraciones? ¿Qué pruebas se exigen? ¿Qué estándares de código se aplican?	Revisión por pares y CI/CD; arquitectura modular para facilitar cambios.
Negocio	¿Qué valor aporta a la universidad? ¿Qué metas debe apoyar? ¿Cómo se mide el éxito?	Soporte a la economía circular estudiantil; indicadores de uso y transacciones.
Innovación	¿Qué diferencia al sistema de un grupo de mensajería? ¿Qué funciones pueden explorarse? ¿Cómo se valida una mejora?	Mecanismos de reputación y recomendación; experimentación con feedback de usuarios.
Sociedad	¿Cómo impacta en la comunidad? ¿Qué riesgos de fraude o abuso existen? ¿Qué necesidades de inclusión aparecen?	Políticas de privacidad y moderación; diseño accesible para distintos perfiles.
Técnica	¿Qué dispositivos y navegadores deben soportarse? ¿Qué rendimiento mínimo se exige? ¿Qué disponibilidad se requiere?	Compatibilidad multiplataforma; cifrado, copias de seguridad y alta disponibilidad.

Las perspectivas de contexto permiten analizar el marketplace universitario desde diferentes enfoques para identificar requisitos, restricciones y riesgos que podrían pasar desapercibidos si el sistema se analiza únicamente desde una visión funcional. Cada perspectiva aporta preguntas específicas relacionadas con usuarios, procesos, tecnología, negocio y entorno social, facilitando una comprensión más completa del sistema y su interacción con el contexto universitario.

En conjunto, estas perspectivas ayudan a mejorar la trazabilidad de requisitos y permiten detectar conflictos entre necesidades técnicas, organizacionales y operativas. Además, facilitan la toma de decisiones durante el desarrollo porque permiten evaluar cómo una modificación funcional puede impactar en la seguridad, el rendimiento, la experiencia del usuario o las políticas institucionales.

3.3 Declaración de Visión

Para estudiantes universitarios que necesitan comprar, vender o intercambiar materiales académicos, el marketplace universitario es una plataforma web colaborativa que centraliza publicaciones, comunicación y reputación para facilitar transacciones seguras y confiables. A diferencia de los grupos informales de mensajería, nuestro producto organiza la oferta por categorías, precios y estado de los productos en un solo lugar.

3.4 Límite del Sistema

El límite del sistema separa lo que el equipo de desarrollo controla directamente de los elementos externos que solo pueden integrarse o consultarse. En el marketplace, dentro del límite quedan la gestión de usuarios, publicaciones, búsqueda, mensajería, reputación, reservas y panel administrativo. Fuera del límite quedan las normas universitarias, los servicios externos de pago, los sistemas de correo o notificación y las decisiones disciplinarias de la institución.

La siguiente tabla resume las entidades externas relevantes para definir la frontera del sistema y comprender las interfaces principales.

Tabla 2. Limite del sistema

Entidad externa	Datos intercambiados	Protocolo / medio	Frecuencia	Observación
Estudiantes	Perfil, publicaciones, mensajes, reservas y calificaciones	HTTPS / WebSocket	Continuo y bajo demanda	Principal fuente de interacción del sistema
Administrador del marketplace	Reportes, bloqueos, aprobaciones y métricas	HTTPS / panel web	Diario y ante incidentes	Controla la moderación
Pasarela de pago	Token de pago, confirmación y estado de la transacción	API REST sobre HTTPS	Por transacción	Servicio externo de terceros
Servicio de correo/notificaciones	Confirmaciones, alertas y recuperación de cuentas	SMTP o API HTTPS	Por evento	Apoya la comunicación asíncrona
Almacenamiento externo de archivos	Imágenes y adjuntos de publicaciones	API S3 / HTTPS	Por carga y consulta	Optimiza almacenamiento y distribución

3.5 Mapa Conceptual Integrador

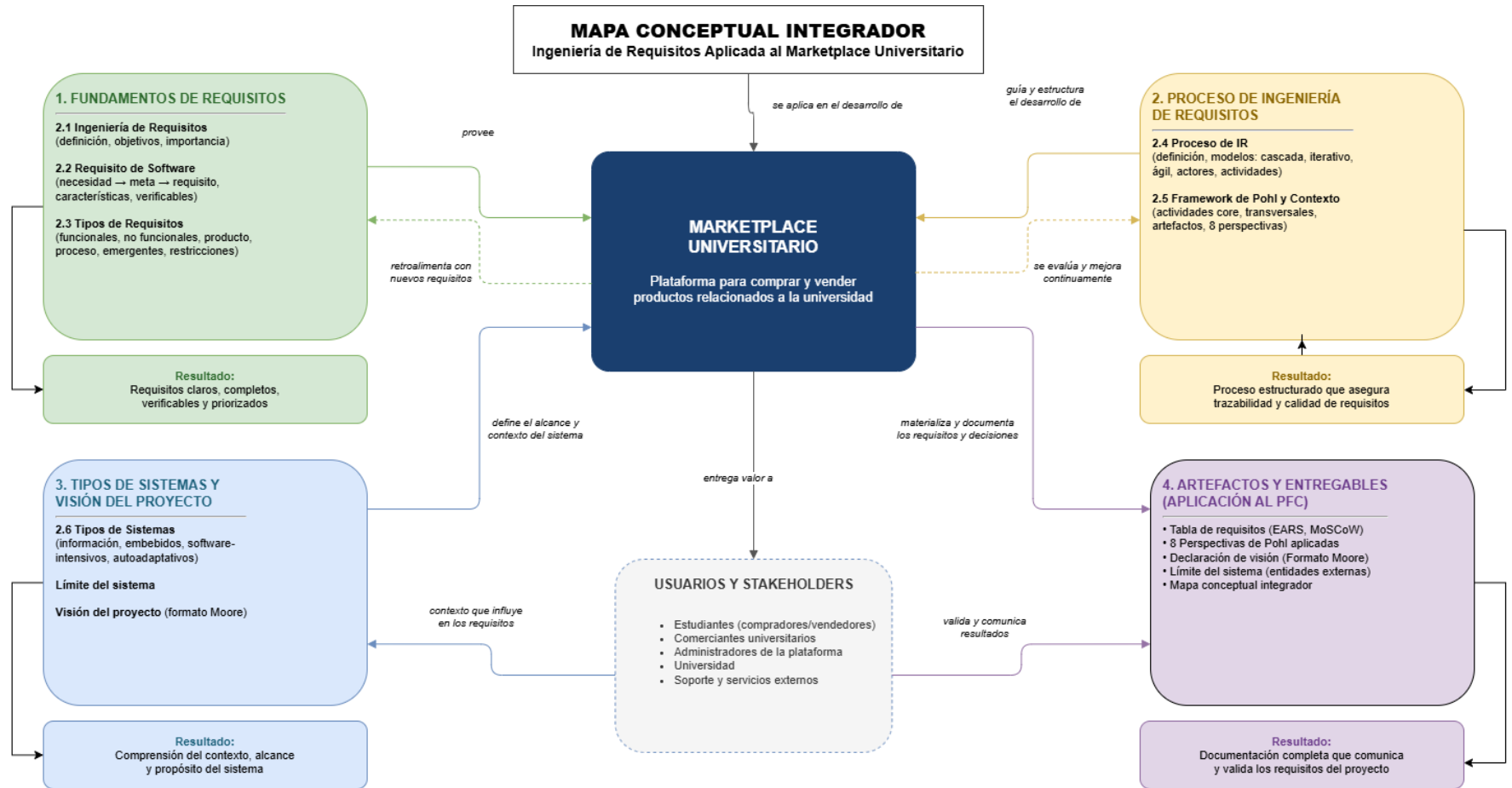


Ilustración 1. Mapa conceptual integrador de Ingeniería de Requisit

4 CONCLUSIONES

Conclusión 1

La ingeniería de requisitos es la base que permite transformar necesidades difusas en especificaciones claras y verificables. En el marketplace universitario, esta disciplina asegura que la solución responda a problemas reales como la dispersión de publicaciones, la falta de confianza entre usuarios y la necesidad de organizar materiales académicos. Su valor no se limita a redactar documentos, sino a reducir retrabajos y mejorar la comunicación entre el equipo y los usuarios. Por ello, su aplicación fortalece tanto la calidad del producto como la viabilidad del proyecto [1,6].

Conclusión 2

El uso de requisitos cuantificables mejora la posibilidad de probar el sistema y tomar decisiones técnicas objetivas. En este proyecto, definir tiempos máximos de respuesta, reglas de seguridad y condiciones de uso ayuda a que el equipo desarrolle una plataforma medible y no basada en impresiones subjetivas. Esto facilita la validación con usuarios y convierte los objetivos de calidad en metas concretas. En consecuencia, la documentación precisa de requisitos eleva la confiabilidad del marketplace y simplifica su mantenimiento [1,5].

Conclusión 3

La correcta clasificación de los requisitos permite ordenar el alcance del sistema y evitar confusiones entre negocio, usuario, funcionalidad y calidad. Para el marketplace universitario, distinguir esos grupos hace posible diseñar una solución coherente, donde cada necesidad tenga una forma clara de expresarse y verificarse. Además, esta estructura facilita la trazabilidad y el control de cambios durante el desarrollo. En términos académicos y profesionales, el ejercicio demuestra que una buena ingeniería de requisitos mejora la gestión del proyecto y la calidad del software entregado [1,4].

Conclusión 4

La ingeniería de requisitos resulta esencial porque reduce la posibilidad de construir un sistema que no responda a las necesidades reales de los usuarios. En el marketplace universitario, este proceso permite definir con claridad qué información debe contener cada publicación, cómo se valida la reputación y qué reglas gobiernan la interacción entre compradores y vendedores. Al documentar y validar los requisitos desde el inicio, el proyecto disminuye retrabajos y mejora la coherencia técnica. Además, fortalece la toma de decisiones del equipo porque cada funcionalidad se conecta con una necesidad concreta [6].

Conclusión 5

Un requisito de software bien formulado debe ser claro, verificable y medible para evitar interpretaciones distintas entre los integrantes del equipo y los usuarios. En este proyecto, eso implica traducir ideas generales como “seguridad” o “rapidez” en condiciones observables, como tiempos máximos de respuesta o mecanismos concretos de protección de datos. Esta práctica mejora la calidad de la especificación y facilita las pruebas posteriores. Por ello, redactar requisitos con precisión no es un detalle formal, sino una condición para construir un producto útil y evaluable [5].

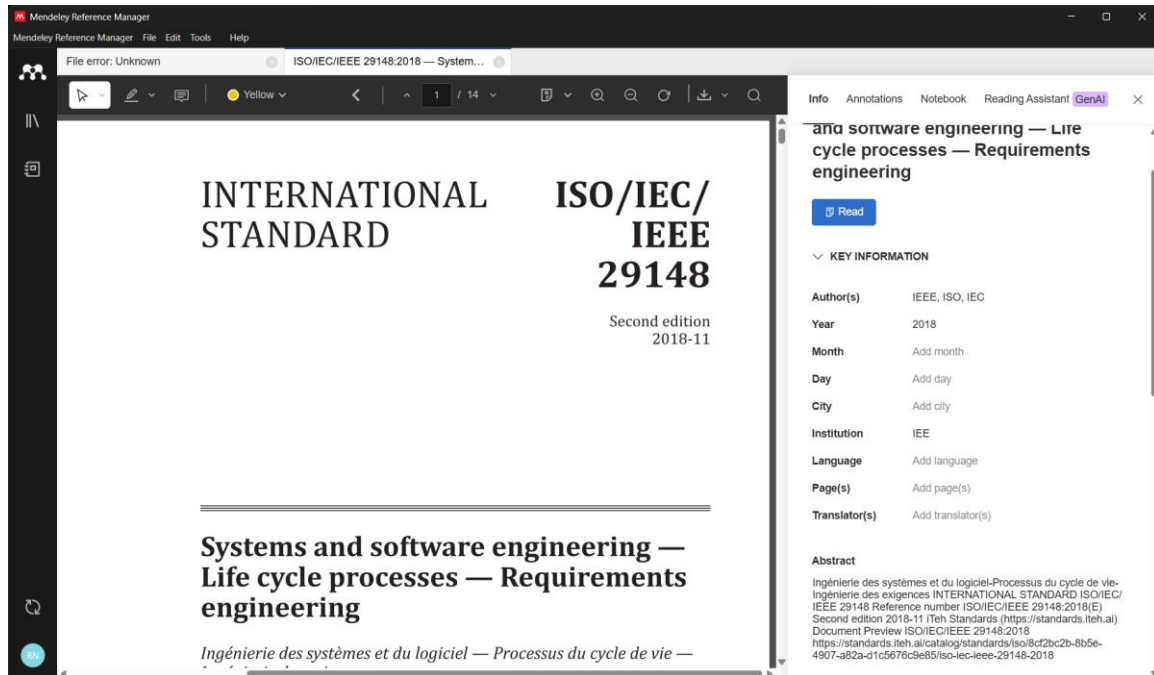
5 REFERENCIAS

- [1] IEEE, ISO, and IEC, “ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering,” 2018. doi: 10.1109/IEEESTD.2018.8559686.
- [2] Stan Böhne, Martin Glinz, Hans van Loenhoud, and Stefan Staal, “CPRE Foundation Level Syllabus,” Feb. 2024. Accessed: May 23, 2026. [Online]. Available: <https://ireb.org/de>
- [3] Hironori Washizaki, Ed., *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0*, 4.0. IEEE Computer Society, 2024.
- [4] IEEE, IEC, and ISO, *ISO/IEC/IEEE 15288:2023 — Systems and software engineering — System life cycle processes*. IEEE, 2023.
- [5] ISO and IEC, “ISO/IEC 25010:2023 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model,” 2023.
- [6] Klaus Pohl and Chris Rupp, “Requirements Engineering Fundamentals: A Study Guide for the Certified Professional for Requirements Engineering Exam, Foundation Level, IREB compliant,” vol. 2nd edition, pp. 1–183, 2015, doi: 10.1007/978-3-662-69204-2.
- [7] IREB e.V. (International Requirements Engineering Board), “Certified Professional for Requirements Engineering (CPRE) Nivel Básico - Programa de estudios,” *IREB e.V. (International Requirements Engineering Board)*, pp. 1–51, Feb. 2024, Accessed: May 23, 2026. [Online]. Available: <https://www.ireb.org/>
- [8] Hironori Washizaki, “Guide to the Software Engineering Body of Knowledge (SWEBOK Guide),” *IEEE Computer Society*, pp. 1–386, Sep. 2025, Accessed: May 23, 2026. [Online]. Available: <https://www.computer.org/swebok>

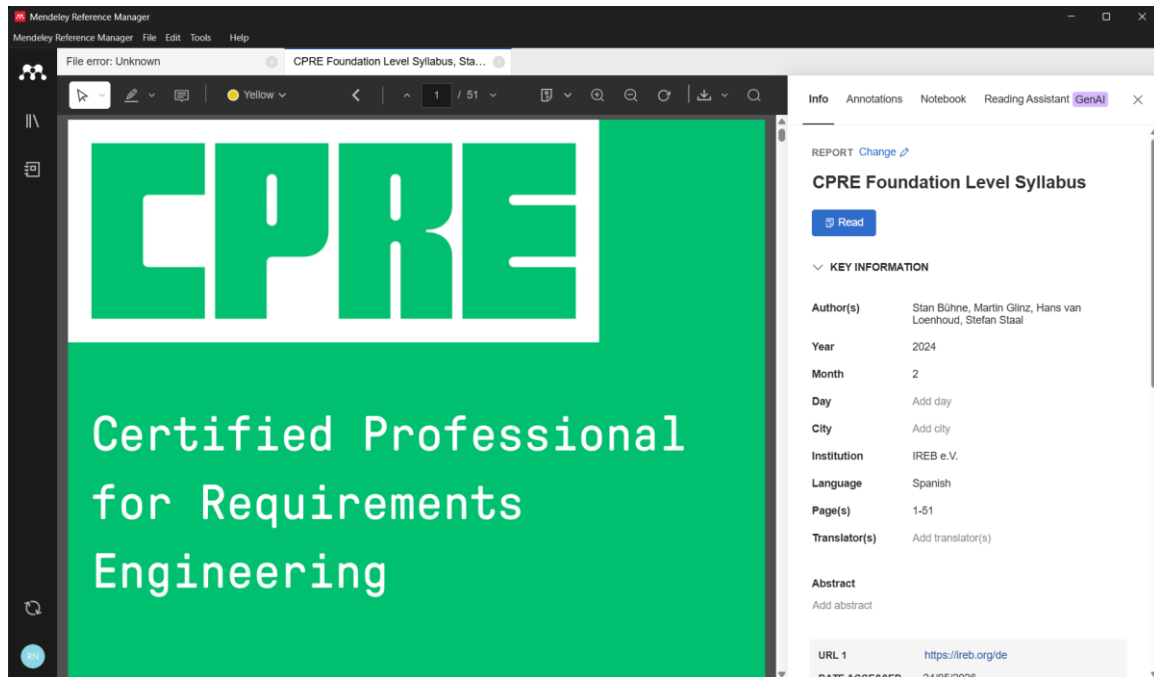
- [9] ISO/IEC/IEEE, “Systems and software engineering — Software life cycle processes,” Geneva, Switzerland, 2017. doi: 10.1109/IEEESTD.2017.8221991.

6ANEXOS

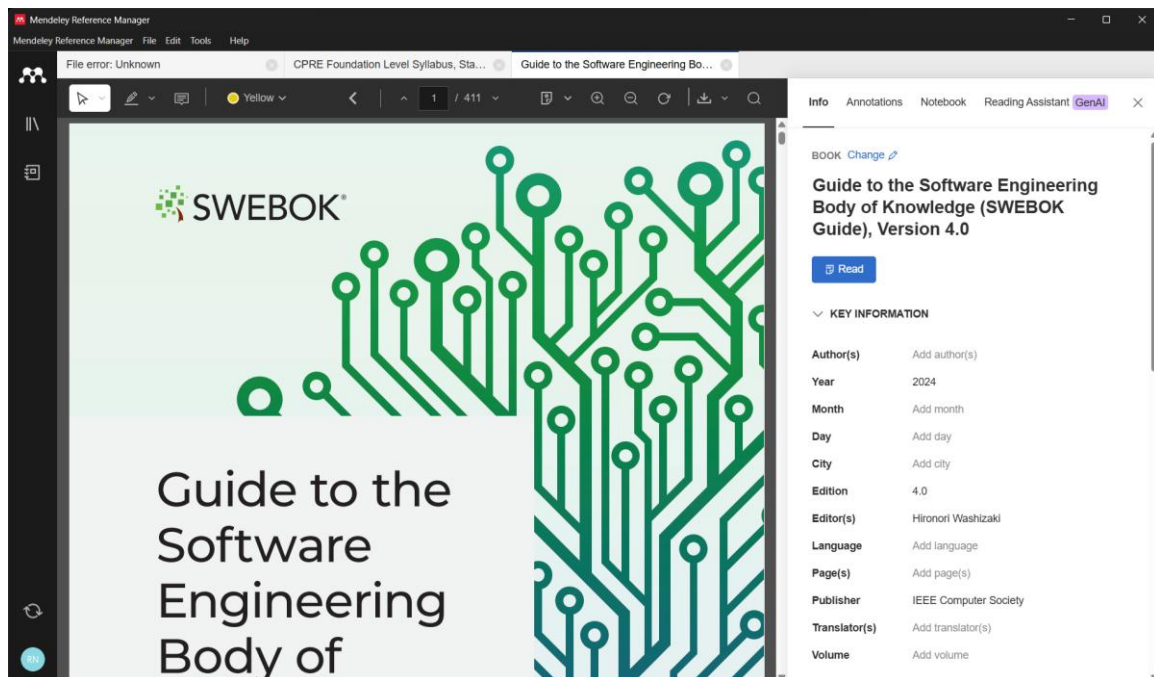
Anexo 1: Exportación IEEE desde Mendeley.



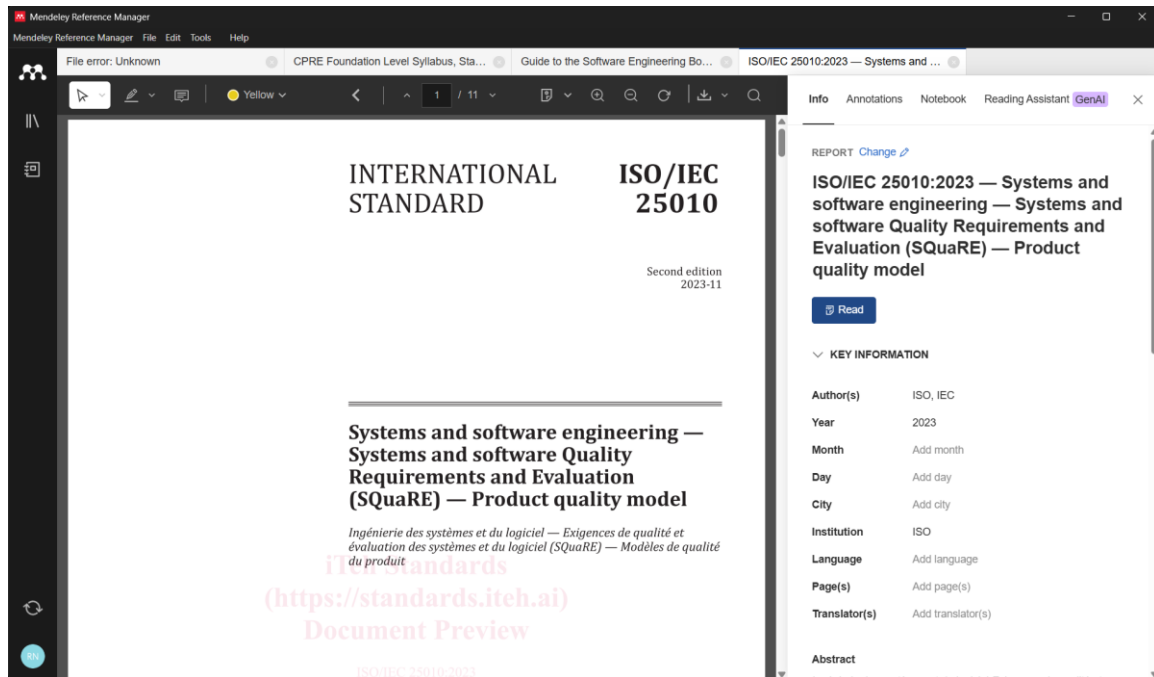
Anexo 2: Exportación IEEE desde Mendeley.



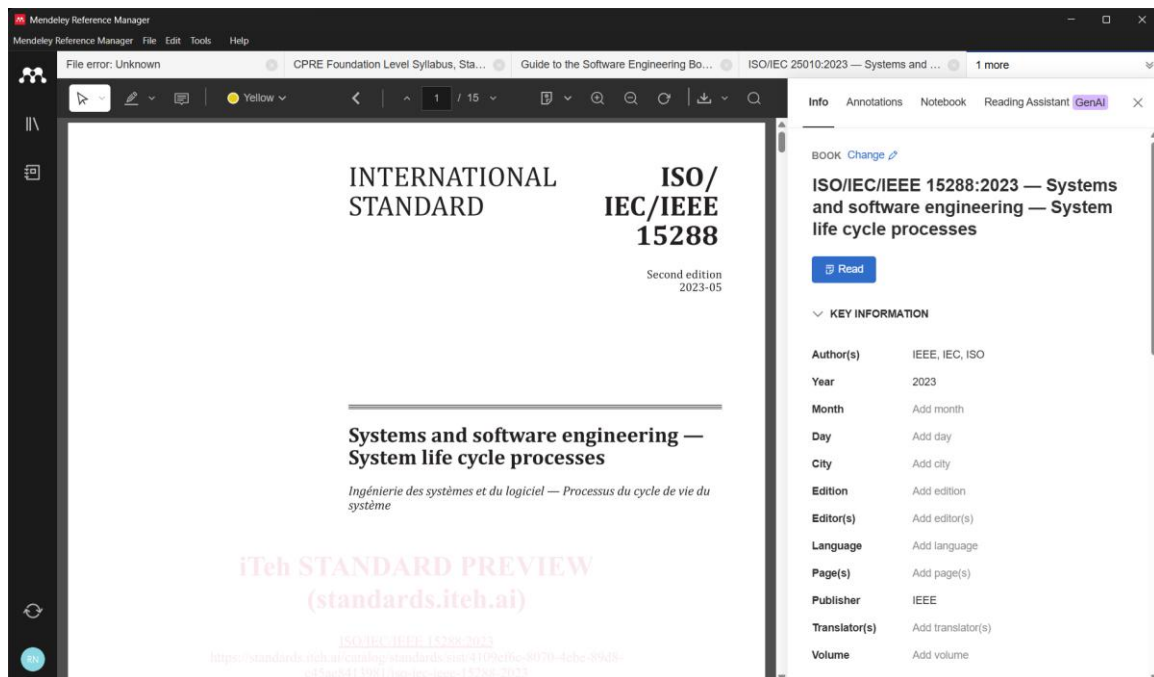
Anexo 3: Exportación IEEE desde Mendeley.



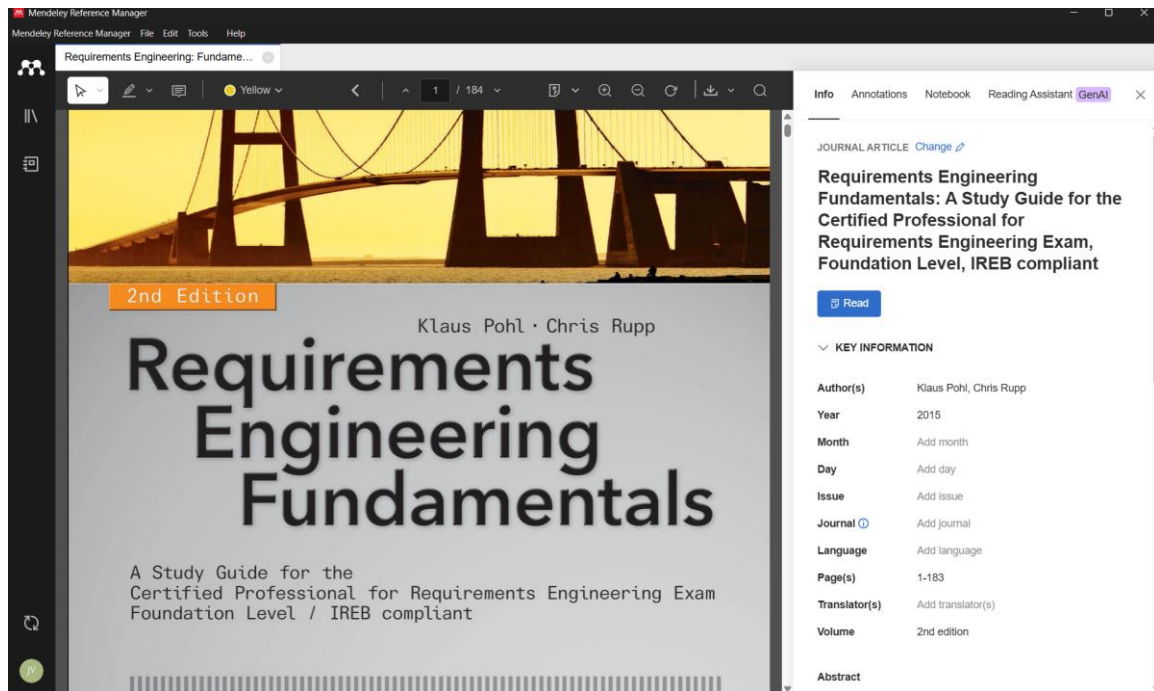
Anexo 4: Exportación IEEE desde Mendeley.



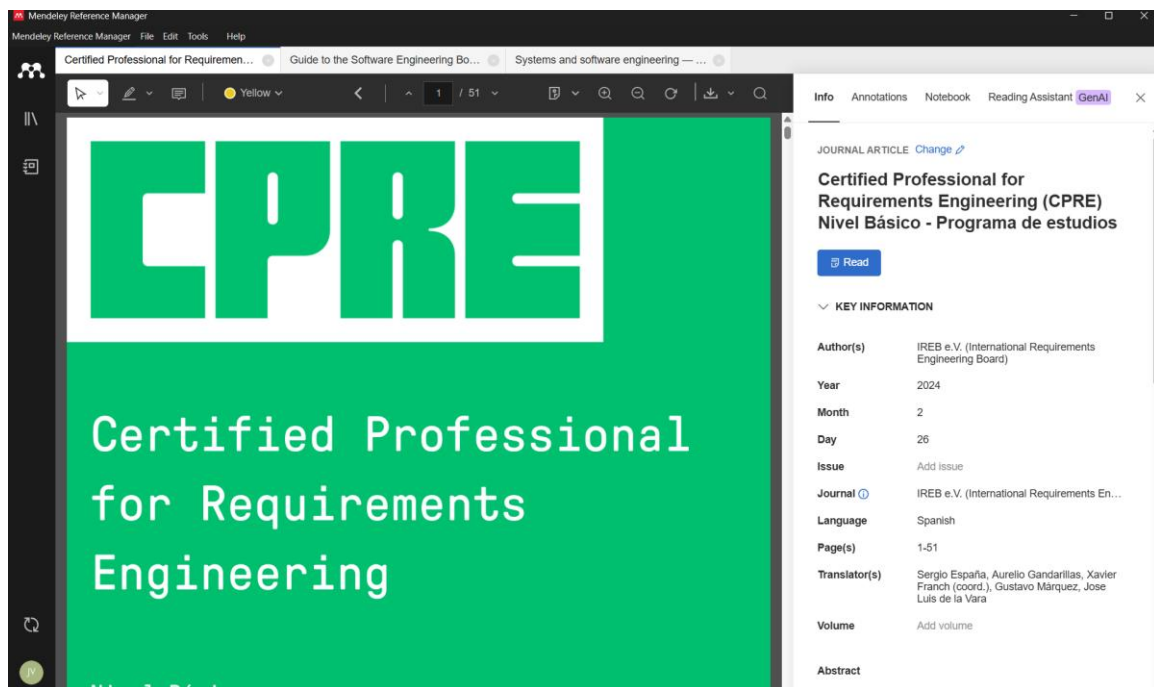
Anexo 5: Exportación IEEE desde Mendeley.



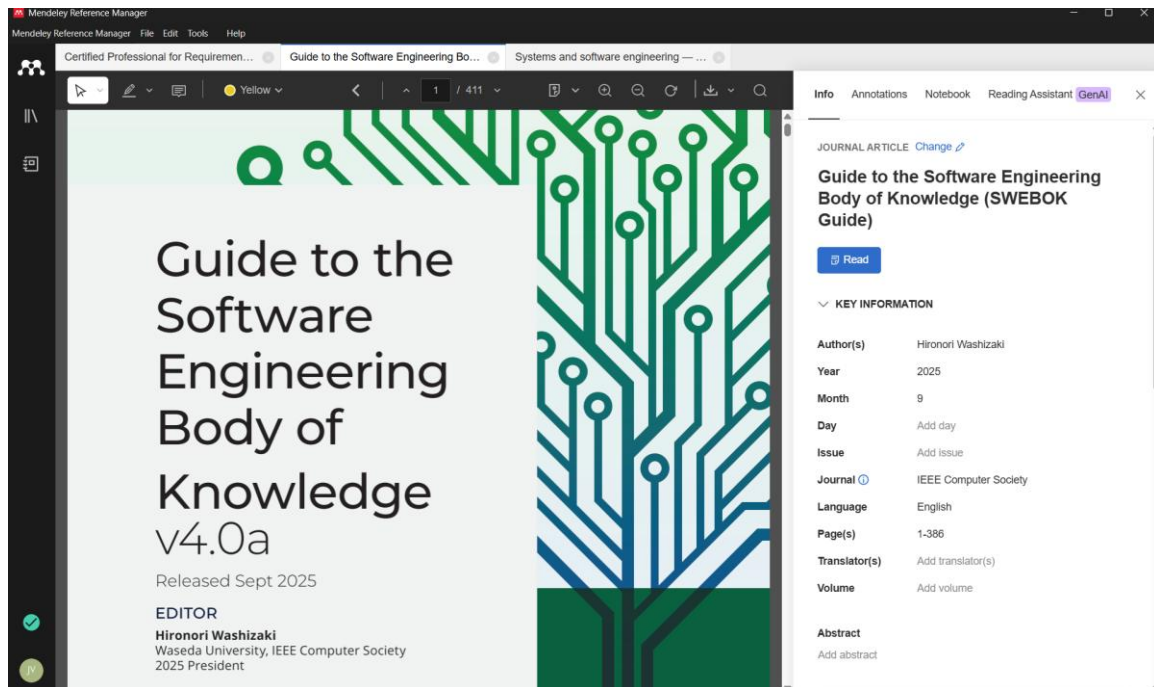
Anexo 6: Exportación IEEE desde Mendeley.



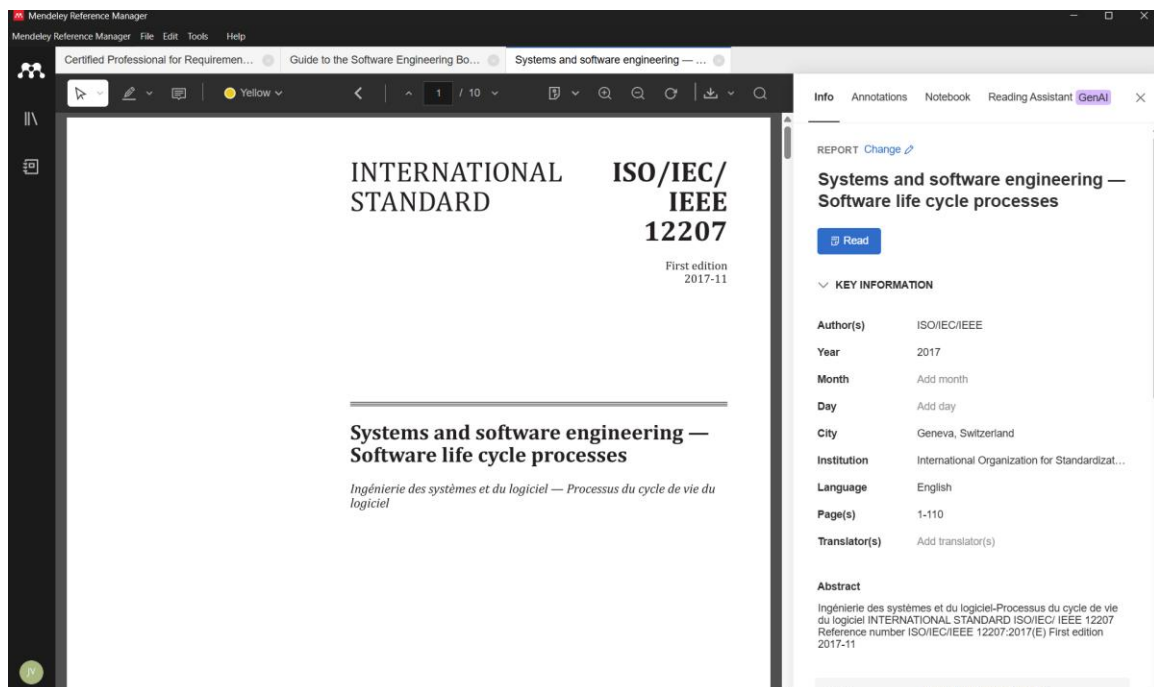
Anexo 7: Exportación IEEE desde Mendeley.



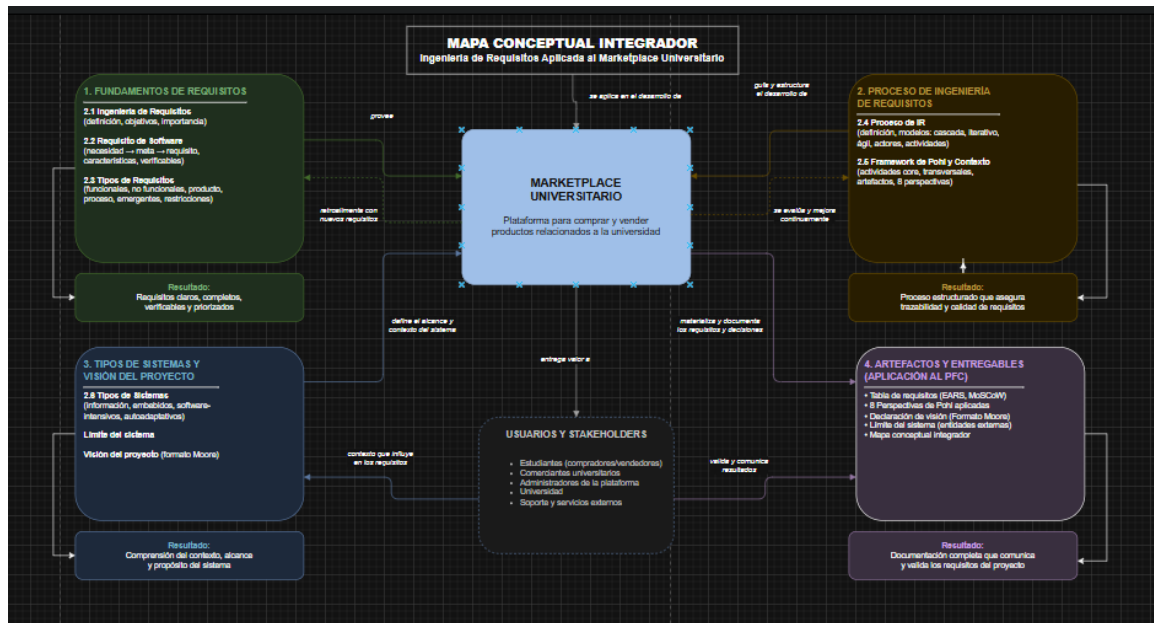
Anexo 8: Exportación IEEE desde Mendeley.



Anexo 9: Exportación IEEE desde Mendeley.



Anexo 10: Mapa Conceptual en Draw.io



7 DECLARACIÓN DE USO DE LA IA

Durante la elaboración de esta práctica experimental utilizamos herramientas de IA como apoyo en tareas de redacción, organización de ideas, revisión gramatical y estructuración del contenido académico. Sin embargo, el análisis, selección de información, interpretación de conceptos, desarrollo de requisitos y adaptación al proyecto fueron realizados y revisados por los integrantes del grupo. El uso de IA se empleó únicamente como herramienta de apoyo complementaria y no sustituyó el criterio académico ni el trabajo colaborativo desarrollado en la práctica experimental.